Tugas Praktikum Pemrograman Mobile
Pertemuan Ke - 5



Nama : **Michael Cornelius Bong**
NIM : **20240801004**
Jurusan : **Ilmu Komputer**
Prodi : **Teknik Informatika**

Universitas Esa Unggul

Praktikum Flutter

1. Tujuan

Praktikum 5 bertujuan agar kita memahami cara menyimpan data secara lokal pada perangkat. Kita mempelajari pemilihan penyimpanan berdasarkan jenis - jenis datanya, menggunakan shared_preferences untuk pengaturan sederhana, menggunakan SQLite dengan sqflite untuk data yang lebih terstruktur, menerapkan CRUD, serta memastikan data tetap tersimpan saat aplikasi ditutup.

2. Alat dan Bahan

- Komputer Atau Laptop
- Flutter Package (Flutter - Install)
- Android Studio / IDE Dengan Support Flutter & Dart
- Emulator SDK Android / HP Android Dengan Debugging Mode
- Koneksi Internet Untuk Mengunduh Package

3. Percobaan Pembuatan Database Lokal

3.1 Pengaturan Sederhana SharedPreferences :

```
PS C:\Users\Asus\Documents\University\Semester 5\Pemrograman Mobile\Praktikum\praktikum_lima> flutter pub add shared_preferences
Resolving dependencies...
Downloading packages... (6.2s)
  cupertino_icons 1.0.9 (2.0.0 available)
+ ffi 2.2.0
+ file 7.0.1
+ flutter_web_plugins 0.0.0 from sdk flutter
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ shared_preferences 2.5.6
+ shared_preferences_android 2.4.28
+ shared_preferences_foundation 2.5.7
+ shared_preferences_linux 2.4.1
+ shared_preferences_platform_interface 2.4.2
+ shared_preferences_web 2.4.3
+ shared_preferences_windows 2.4.1
  test_api 0.7.12 (0.7.14 available)
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
+ xdg_directories 1.1.0
Changed 17 dependencies!
```

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const PengaturanPage(),
    );
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
```

```
    _gelap = prefs.getBool('gelap') ?? false;
    _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
  });
  await prefs.setInt('bukaKe', _bukaKe);
}

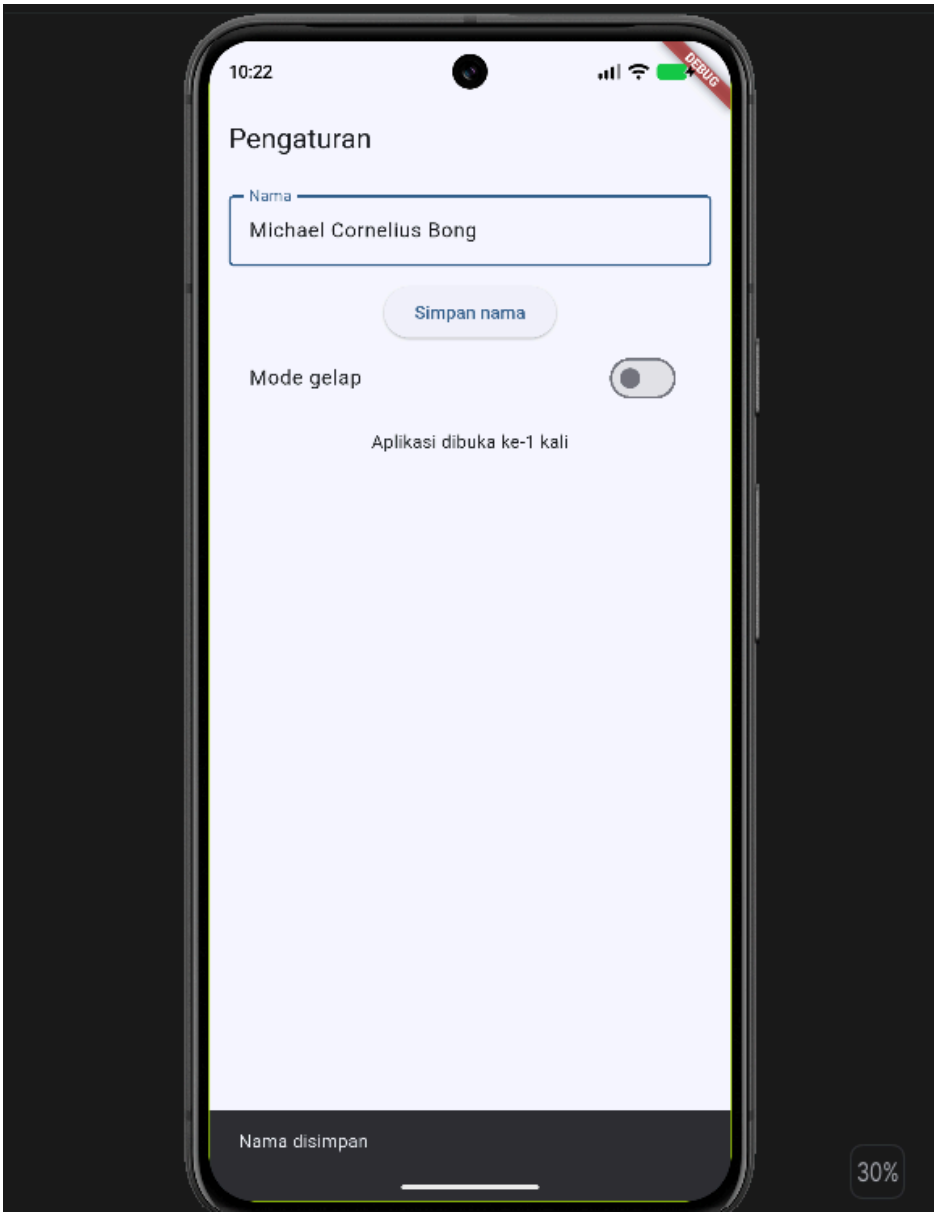
Future<void> _simpanNama() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('nama', _controller.text.trim());
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Nama disimpan')),
  );
}

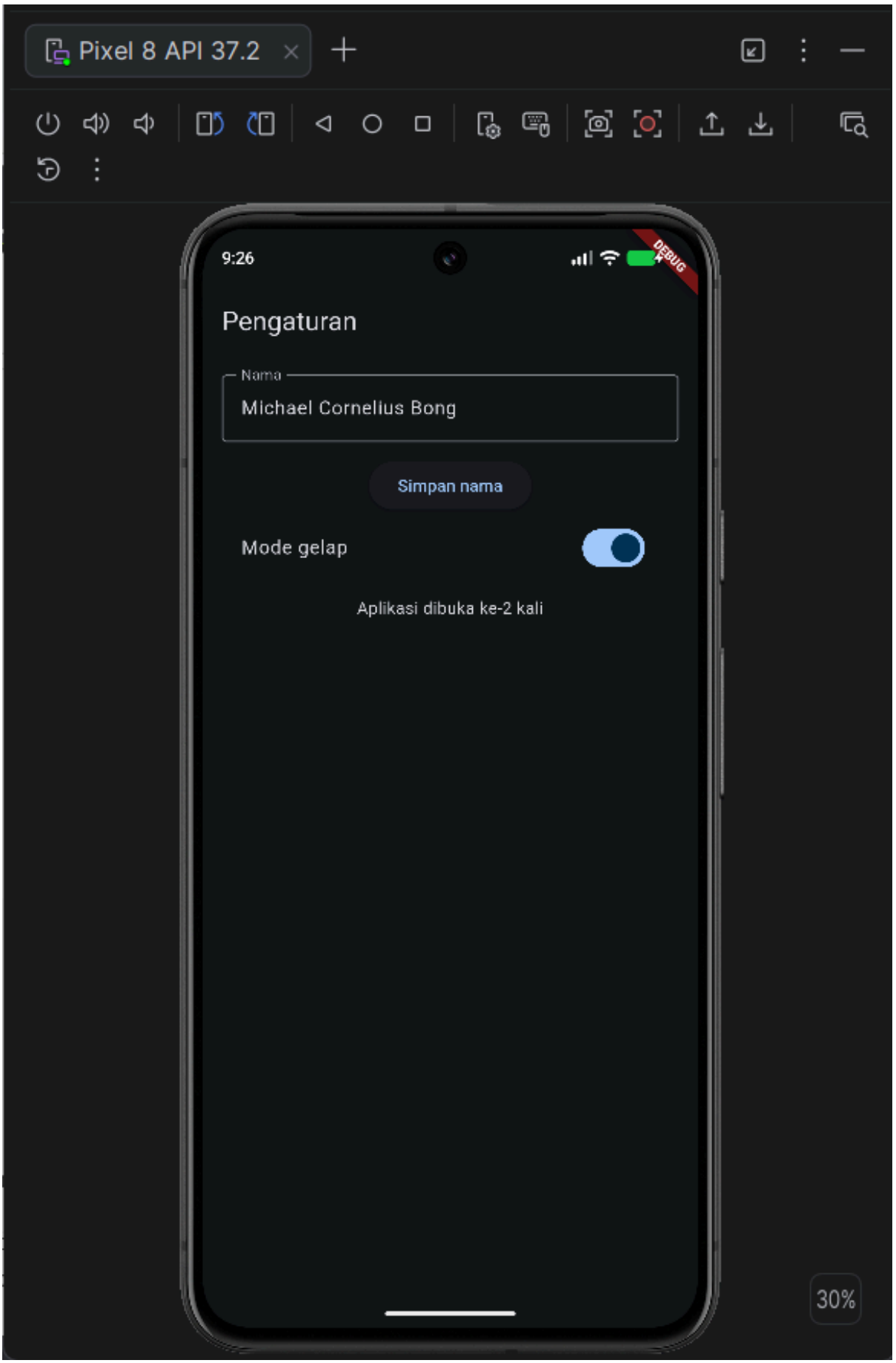
Future<void> _ubahTema(bool nilai) async {
  setState(() => _gelap = nilai);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);
}

@override
Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(
      brightness: _gelap ? Brightness.dark : Brightness.light,
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    child: Scaffold(
```

```
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
```

```
90                onPressed: _simpanNama,
91                child: const Text('Simpan nama'),
92              ),
93              SwitchListTile(
94                title: const Text('Mode gelap'),
95                value: _gelap,
96                onChanged: _ubahTema,
97              ),
98              const SizedBox(height: 12),
99              Text('Aplikasi dibuka ke-$_bukaKe kali'),
100           ],
101         ),
102       ),
103     ),
104   );
105 }
106 }
```




Pixel 8 API 37.2
9:26
DEBUG
Pengaturan
Nama
Michael Cornelius Bong
Simpan nama
Mode gelap
Aplikasi dibuka ke-2 kali
30%

3.2 Mempersiapkan Database Dengan SQLite :

```
PS C:\Users\Asus\Documents\University\Semester 5\Pemrograman Mobile\Praktikum\praktikum_lima\android> flutter pub add sqflite path
"sqflite" is already in "dependencies". Will try to update the constraint.
"path" is already in "dependencies". Will try to update the constraint.
Resolving dependencies in `C:\Users\Asus\Documents\University\Semester 5\Pemrograman Mobile\Praktikum\praktikum_lima`...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
  vector_math 2.4.0 (2.4.3 available)
```

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
  final int? id;
  final String judul;
  final String isi;
  const Catatan({this.id, required this.judul, required this.isi});
  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};
  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
  );
}

class DbHelper {
  static Database? _db;
  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
```

```
                'judul TEXT NOT NULL, '
                'isi TEXT NOT NULL)',
            );
          },
      );
      return _db!;
    }
    static Future<int> tambah(Catatan c) async {
      final db = await database;
      return db.insert('catatan', c.toMap());
    }
    static Future<List<Catatan>> semua() async {
      final db = await database;
      final rows = await db.query('catatan', orderBy: 'id DESC');
      return rows.map(Catatan.fromMap).toList();
    }
    static Future<int> ubah(Catatan c) async {
      final db = await database;
      return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
    }
    static Future<int> hapus(int id) async {
      final db = await database;
      return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
    }
}

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const PengaturanPage(),
    );
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
```

```dart
final _controller = TextEditingController();
bool _gelap = false;
int _bukaKe = 0;

@override
void initState() {
  super.initState();
  _muat();
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

Future<void> _muat() async {
  final prefs = await SharedPreferences.getInstance();
  if (!mounted) return;
  setState(() {
    _controller.text = prefs.getString('nama') ?? '';
    _gelap = prefs.getBool('gelap') ?? false;
    _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
  });
  await prefs.setInt('bukaKe', _bukaKe);
}

Future<void> _simpanNama() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('nama', _controller.text.trim());
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Nama disimpan')),
  );
}

Future<void> _ubahTema(bool nilai) async {
  setState(() => _gelap = nilai);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);
}

@override
Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(
      brightness: _gelap ? Brightness.dark : Brightness.light,
      colorSchemeSeed: Colors.blue,
```

```
      useMaterial3: true,
    ), ThemeData
    child: Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ), InputDecoration
            ), TextField
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: _simpanNama,
              child: const Text('Simpan nama'),
            ), ElevatedButton
            SwitchListTile(
              title: const Text('Mode gelap'),
              value: _gelap,
              onChanged: _ubahTema,
            ), SwitchListTile
            const SizedBox(height: 12),
            Text('Aplikasi dibuka ke-$_bukaKe kali'),
          ],
        ), Column
      ), Padding
    ), Scaffold
  ); Theme
  }
}
```

3.3 Pembuatan Halaman Daftar Catatan Serta Create / Update :

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
  final int? id;
  final String judul;
  final String isi;
  const Catatan({this.id, required this.judul, required this.isi});
  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};
  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
  );
}

class DbHelper {
  static Database? _db;
  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'judul TEXT NOT NULL, '
              'isi TEXT NOT NULL)',
        );
      },
    );
    return _db!;
  }
  static Future<int> tambah(Catatan c) async {
    final db = await database;
    return db.insert('catatan', c.toMap());
  }
  static Future<List<Catatan>> semua() async {
    final db = await database;
    final rows = await db.query('catatan', orderBy: 'id DESC');
    return rows.map(Catatan.fromMap).toList();
```

```
  }
  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }
  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const CatatanPage(),
    );
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}
```

```
class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
```

```
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }

  Future<void> _simpanNama() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('nama', _controller.text.trim());
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Nama disimpan')),
    );
  }

  Future<void> _ubahTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return Theme(
      data: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ), ThemeData
      child: Scaffold(
        appBar: AppBar(title: const Text('Pengaturan')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _controller,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ), InputDecoration
              ), TextField
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: _simpanNama,
                child: const Text('Simpan nama'),
              ), ElevatedButton
```

```
                SwitchListTile(
                  title: const Text('Mode gelap'),
                  value: _gelap,
                  onChanged: _ubahTema,
                ), SwitchListTile
                const SizedBox(height: 12),
                Text('Aplikasi dibuka ke-$_bukaKe kali'),
              ],
            ), Column
          ), Padding
        ), Scaffold
      ); Theme
    }
  }

class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});
  @override
  State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;
  @override
  void initState() {
    super.initState();
    _future = DbHelper.semua();
    }
  void _muat() {
```

```
    setState(() {                                                    60
    _future = DbHelper.semua();
    });
}

Future<void> _buka([Catatan? catatan]) async {await Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
);

if (!mounted) return;
  _muat();
}

@override
Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: FutureBuilder<List<Catatan>>(
        future: _future,
        builder: (context, snapshot) {
```

```dart
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Belum ada catatan'));
          }
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final c = data[i];
              return ListTile(
                title: Text(c.judul),
                subtitle: Text(c.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
                onTap: () => _buka(c),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () async {
                    await DbHelper.hapus(c.id!);
                    _muat();
                  },
                ), IconButton
              ); ListTile
            },
          ); ListView.builder
        },
      ), FutureBuilder
```

```dart
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ), FloatingActionButton
    ); Scaffold
  }
}

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});
  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

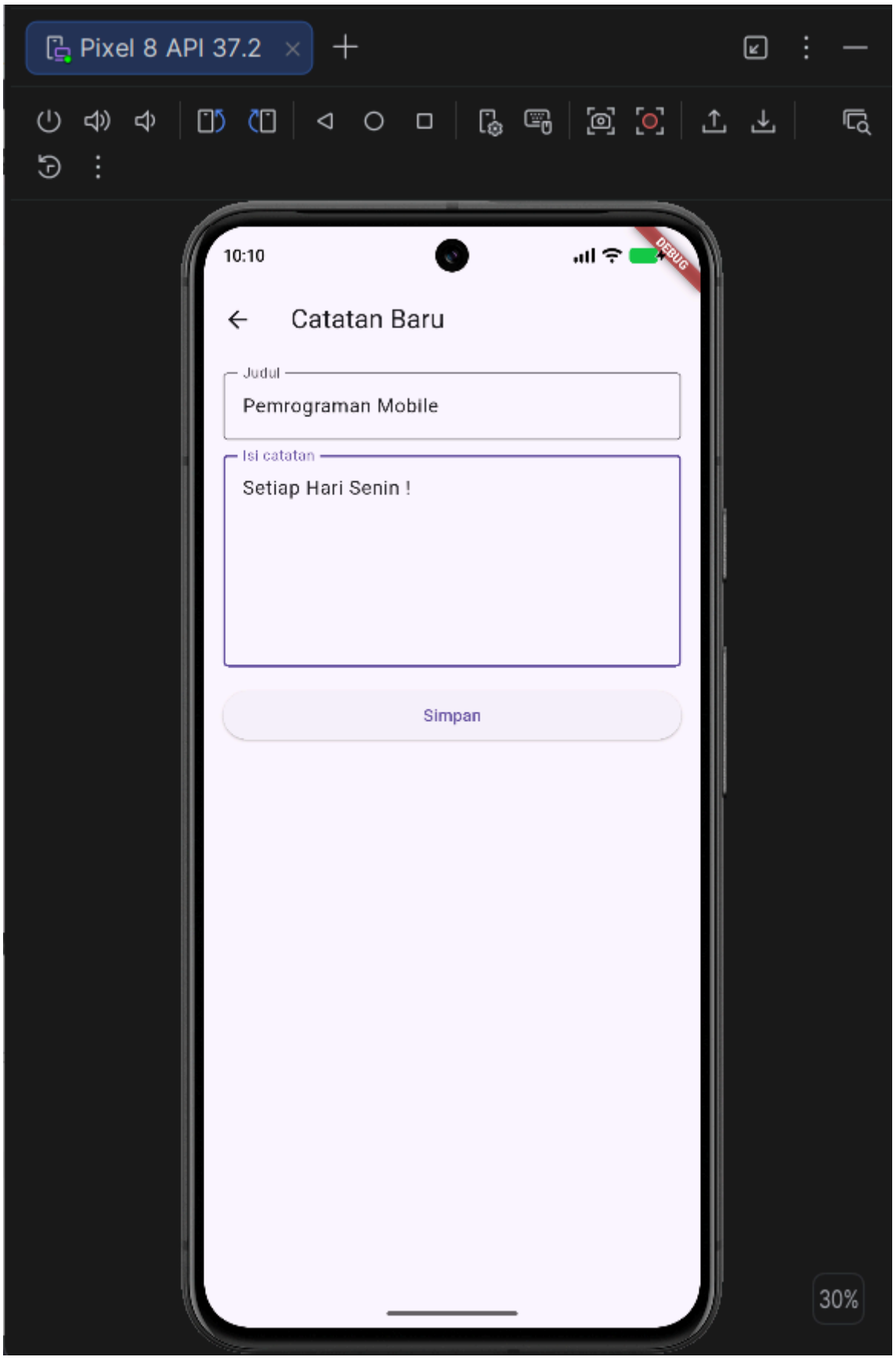
class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;
  @override
```

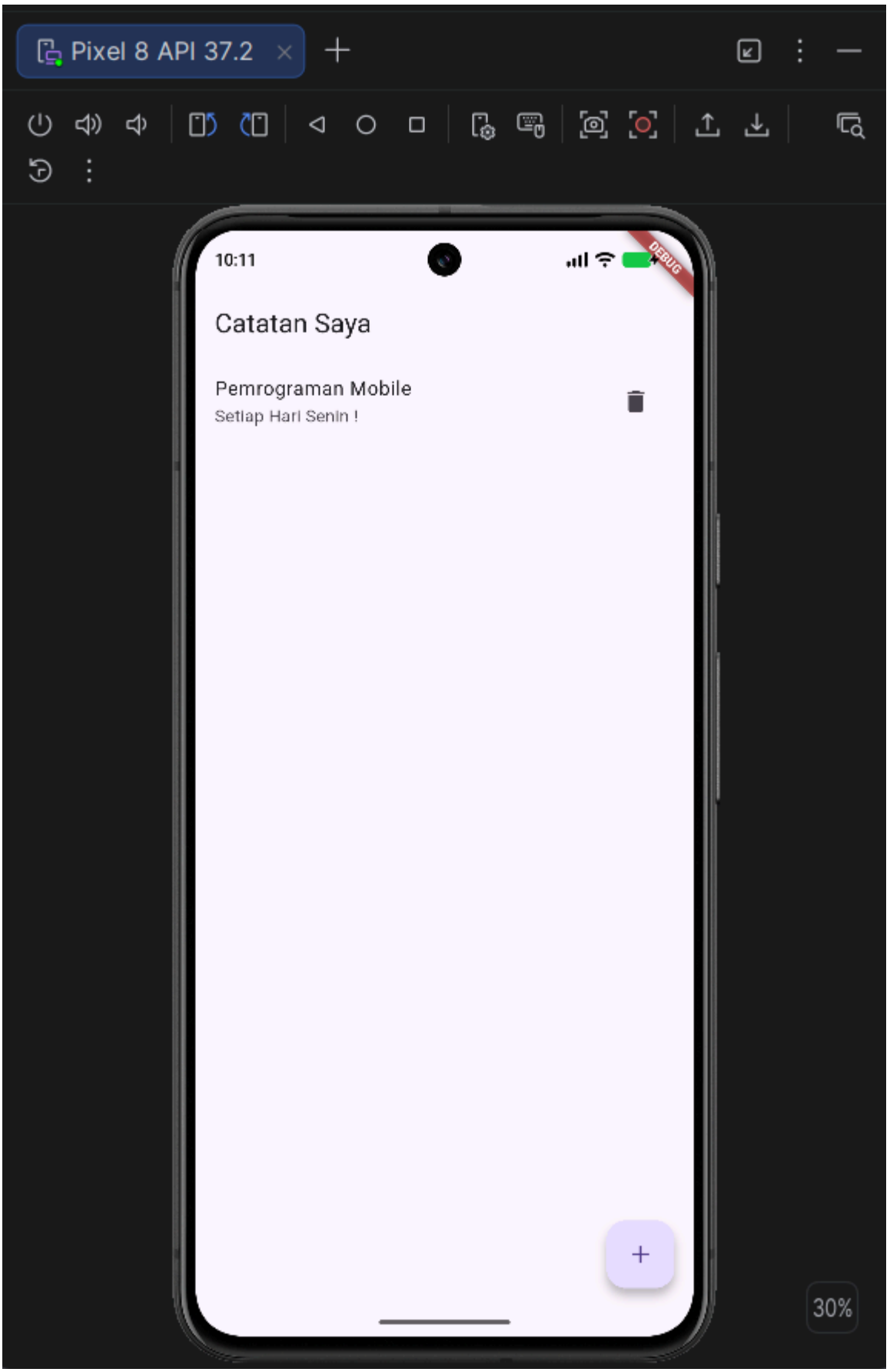
```dart
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }
  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }
  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
    );
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }
  @override
  Widget build(BuildContext context) {
```

```dart
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ), InputDecoration
              validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ), TextFormField
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ), InputDecoration
```

```
299                   validator: (v) =>
300                   (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
301                 ), TextFormField
302                 const SizedBox(height: 16),
303                 ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
304               ],
305             ), ListView
306           ), Form
307         ); Scaffold
308       }
309     }
310
```




Pixel 8 API 37.2
10:10
DEBUG
Catatan Baru
Judul
Pemrograman Mobile
Isi catatan
Setiap Hari Senin !
Simpan
30%


Pixel 8 API 37.2
10:11
DEBUG
Catatan Saya
Pemrograman Mobile
Setiap Hari Senin !
30%

4. Latihan Mandiri

- Tampilkan AlertDialog konfirmasi "Hapus catatan ini?" sebelum data dihapus.
- Tambahkan kolom pencarian yang memfilter daftar memakai where: 'judul LIKE ?', whereArgs: ['%$kata%'].
- Tambahkan kolom dibuat (tanggal dibuat). Naikkan version menjadi 2 dan tambahkan onUpgrade berisi ALTER TABLE catatan ADD COLUMN dibuat TEXT. Amati apa yang terjadi bila version tidak dinaikkan.
- Simpan pilihan urutan (terbaru/terlama) dengan shared_preferences dan terapkan pada orderBy.

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
  final int? id;
  final String judul;
  final String isi;
  final String? dibuat;
  const Catatan({
    this.id,
    required this.judul,
    required this.isi,
    this.dibuat,
  });
  Map<String, Object?> toMap() => {
    'id': id,
    'judul': judul,
    'isi': isi,
    'dibuat': dibuat,
  };
  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
    dibuat: m['dibuat'] as String?,
  );
}

```

```dart
class DbHelper {
  static Database? _db;
  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 2,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'judul TEXT NOT NULL, '
              'isi TEXT NOT NULL, '
              'dibuat TEXT)',
        );
      },
      onUpgrade: (db, oldVersion, newVersion) async {
        if (oldVersion < 2) {
          await db.execute(
            'ALTER TABLE catatan ADD COLUMN dibuat TEXT',
          );
        }
      },
    );
    return _db!;
  }
  static Future<int> tambah(Catatan c) async {
    final db = await database;
    return db.insert('catatan', c.toMap());
  }
  static Future<List<Catatan>> semua({
    String kata = '',
    String orderBy = 'id DESC',
  }) async {
    final db = await database;
    final rows = await db.query(
      'catatan',
      where: kata.isEmpty ? null : 'judul LIKE ?',
      whereArgs: kata.isEmpty ? null : ['%$kata%'],
      orderBy: orderBy,
    );
    return rows.map(Catatan.fromMap).toList();
  }
  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }
  static Future<int> hapus(int id) async {
    final db = await database;
```

```
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const CatatanPage(),
    ); MaterialApp
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;
```

92

```
  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }
```

```dart
  Future<void> _simpanNama() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('nama', _controller.text.trim());
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Nama disimpan')),
    );
  }

  Future<void> _ubahTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }

  @override
  Widget build(BuildContext context) {
    return Theme(
      data: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ), ThemeData
      child: Scaffold(
        appBar: AppBar(title: const Text('Pengaturan')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _controller,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ), InputDecoration
              ), TextField
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: _simpanNama,
                child: const Text('Simpan nama'),
              ), ElevatedButton
              SwitchListTile(
                title: const Text('Mode gelap'),
                value: _gelap,
                onChanged: _ubahTema,
              ), SwitchListTile
              const SizedBox(height: 12),
              Text('Aplikasi dibuka ke-$_bukaKe kali'),
            ],
          ), Column
        ), Padding
      ), Scaffold
    ); Theme
  }
```

```dart
}

class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});
  @override
  State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;
  final _searchController = TextEditingController();
  String _kata = '';
  String _urutan = 'terbaru';

  @override
  void initState() {
    super.initState();
    _muatPengaturan();
  }

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  Future<void> _muatPengaturan() async {
    final prefs = await SharedPreferences.getInstance();
    _urutan = prefs.getString('urutan') ?? 'terbaru';
    _future = DbHelper.semua(
      kata: _kata,
      orderBy: _urutan == 'terbaru' ? 'id DESC' : 'id ASC',
    );
    if (!mounted) return;
    setState(() {});
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua(
        kata: _kata,
        orderBy: _urutan == 'terbaru' ? 'id DESC' : 'id ASC',
      );
    });
  }

  Future<void> _ubahUrutan(String nilai) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('urutan', nilai);
    setState(() {
```

```
      _urutan = nilai;
      _future = DbHelper.semua(
        kata: _kata,
        orderBy: _urutan == 'terbaru' ? 'id DESC' : 'id ASC',
      );
    });
  }

  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
    );

    if (!mounted) return;
    _muat();
  }

  Future<void> _hapus(Catatan catatan) async {
    final konfirmasi = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Konfirmasi'),
        content: const Text('Hapus catatan ini?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Batal'),
          ), TextButton
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Hapus'),
          ), TextButton
        ],
      ), AlertDialog
    );

    if (konfirmasi == true) {
      await DbHelper.hapus(catatan.id!);
      _muat();
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: Column(
        children: [
```

```dart
Padding(
  padding: const EdgeInsets.all(16),
  child: TextField(
    controller: _searchController,
    onChanged: (value) {
      setState(() {
        _kata = value;
        _future = DbHelper.semua(
          kata: _kata,
          orderBy: _urutan == 'terbaru' ? 'id DESC' : 'id ASC',
        );
      });
    },
    decoration: const InputDecoration(
      labelText: 'Cari judul',
      prefixIcon: Icon(Icons.search),
      border: OutlineInputBorder(),
    ), InputDecoration
  ), TextField
), Padding
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: Row(
    children: [
      const Text('Urutan:'),
      const SizedBox(width: 8),
      DropdownButton<String>(
        value: _urutan,
        items: const [
          DropdownMenuItem(
            child: Text('Terbaru'),
          ), DropdownMenuItem
          DropdownMenuItem(
            value: 'terlama',
            child: Text('Terlama'),
          ), DropdownMenuItem
        ],
        onChanged: (value) {
          if (value != null) {
            _ubahUrutan(value);
          }
        },
      ), DropdownButton
    ],
  ), Row
), Padding
Expanded(
  child: FutureBuilder<List<Catatan>>(
```

```
                future: _future,
                builder: (context, snapshot) {
                  if (snapshot.connectionState == ConnectionState.waiting) {
                    return const Center(child: CircularProgressIndicator());
                  }
                  if (snapshot.hasError) {
                    return Center(child: Text('Galat: ${snapshot.error}'));
                  }
                  final data = snapshot.data!;
                  if (data.isEmpty) {
                    return const Center(child: Text('Belum ada catatan'));
                  }
                  return ListView.builder(
                    itemCount: data.length,
                    itemBuilder: (context, i) {
                      final c = data[i];
                      return ListTile(
                        title: Text(c.judul),
                        subtitle: Text(
                          c.isi,
                          maxLines: 2,
                          overflow: TextOverflow.ellipsis,
                        ), Text
                        onTap: () => _buka(c),
                        trailing: IconButton(
                          icon: const Icon(Icons.delete),
                          onPressed: () => _hapus(c),
                        ), IconButton
                      ); ListTile
```

```
                    },
                  ); ListView.builder
                },
              ), FutureBuilder
            ), Expanded
          ],
        ), Column
        floatingActionButton: FloatingActionButton(
          onPressed: () => _buka(),
          child: const Icon(Icons.add),
        ), FloatingActionButton
      ); Scaffold
    }
  }

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
```

```dart
  const FormCatatanPage({super.key, this.catatan});
  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;

  @override
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;

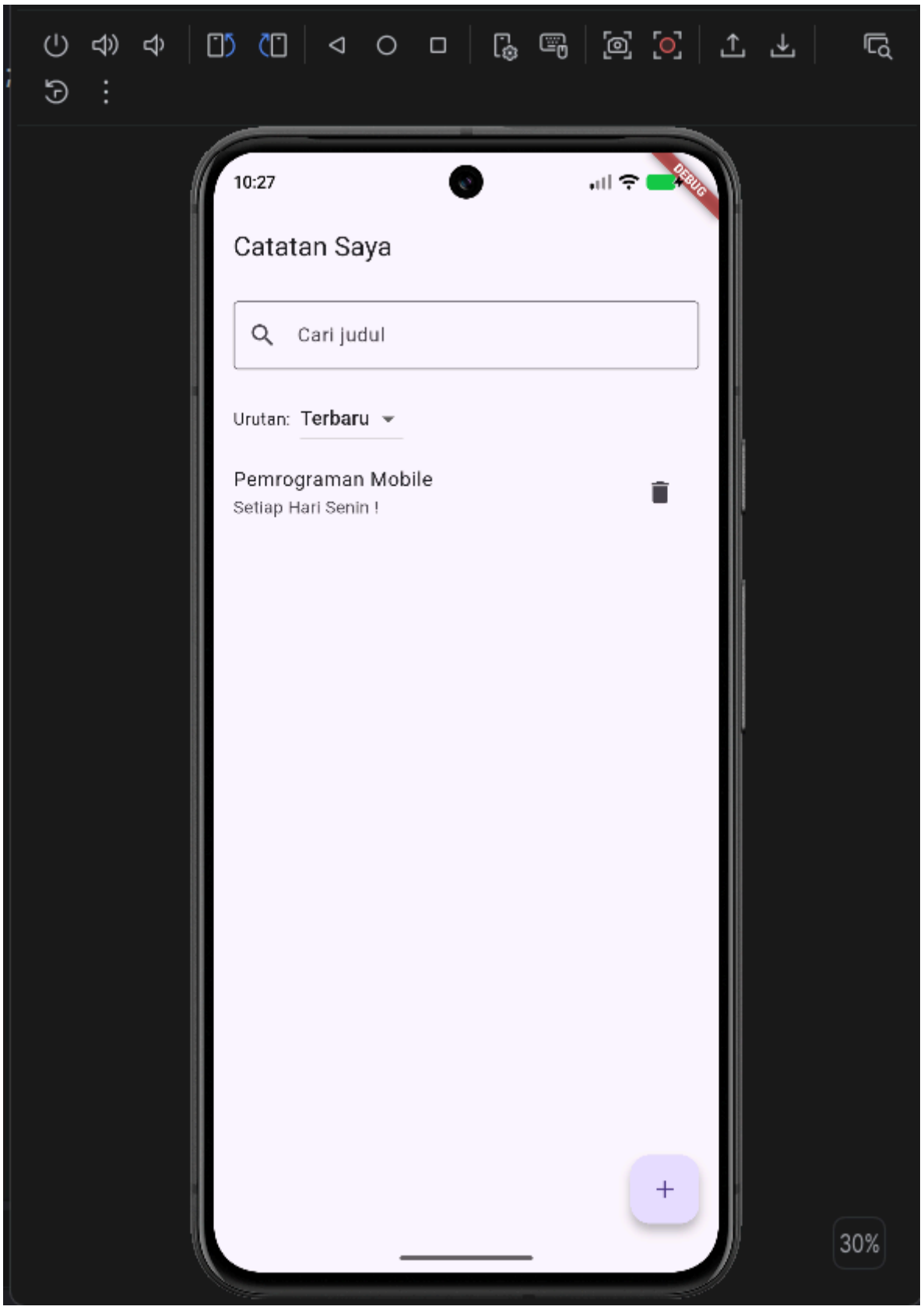
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
      dibuat: widget.catatan?.dibuat ?? DateTime.now().toString(),
    );

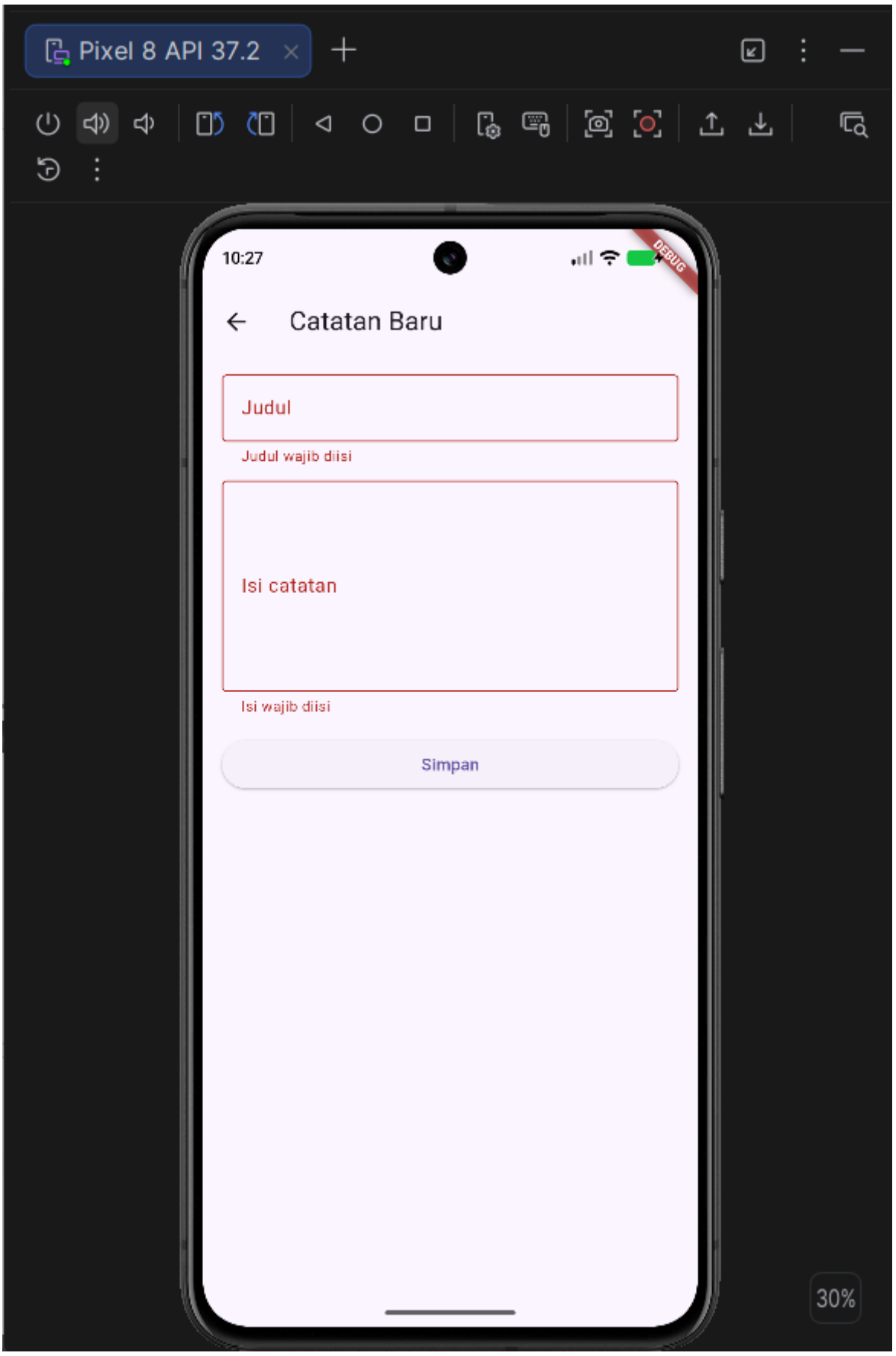
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }

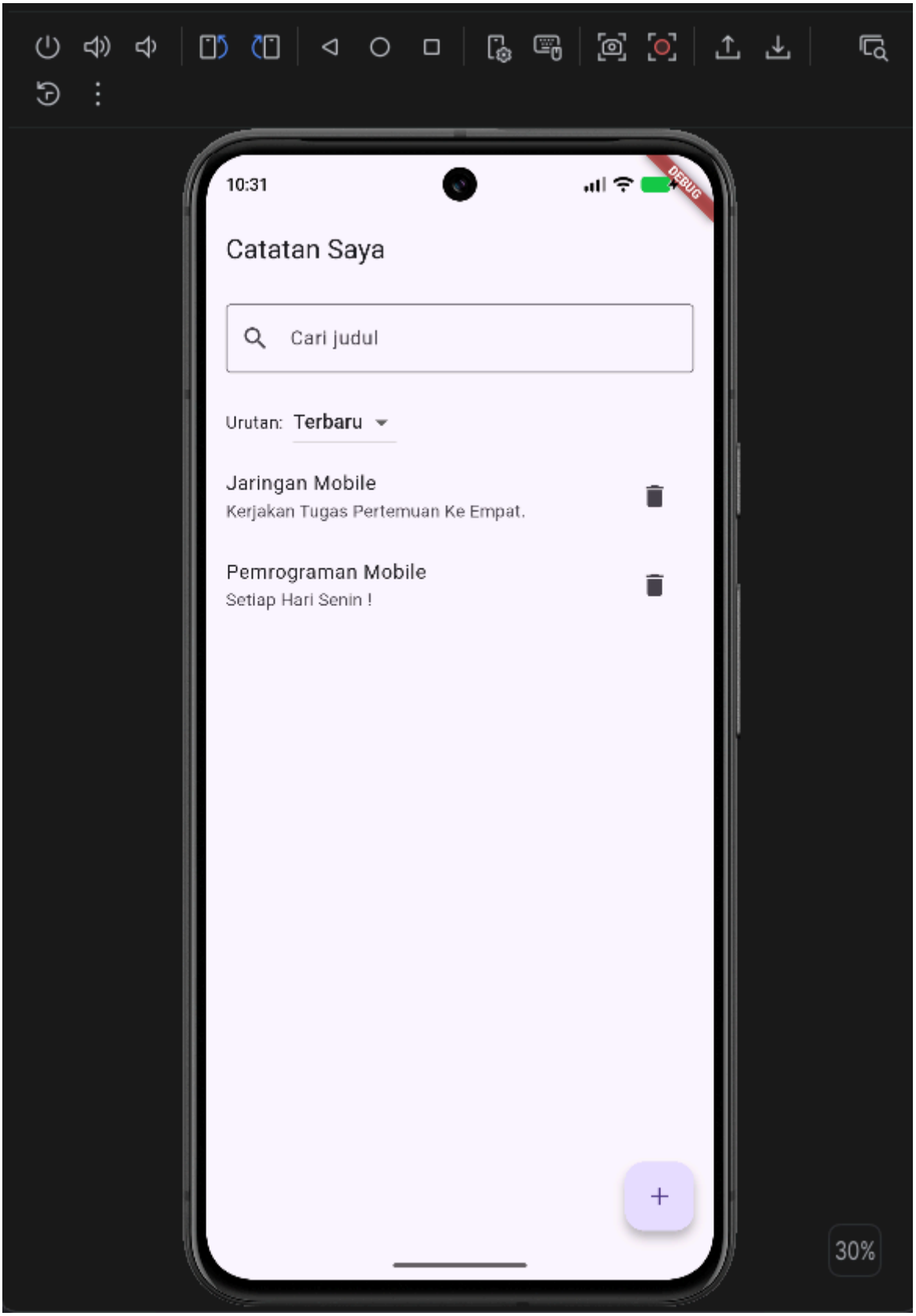
    if (!mounted) return;
    Navigator.pop(context);
  }

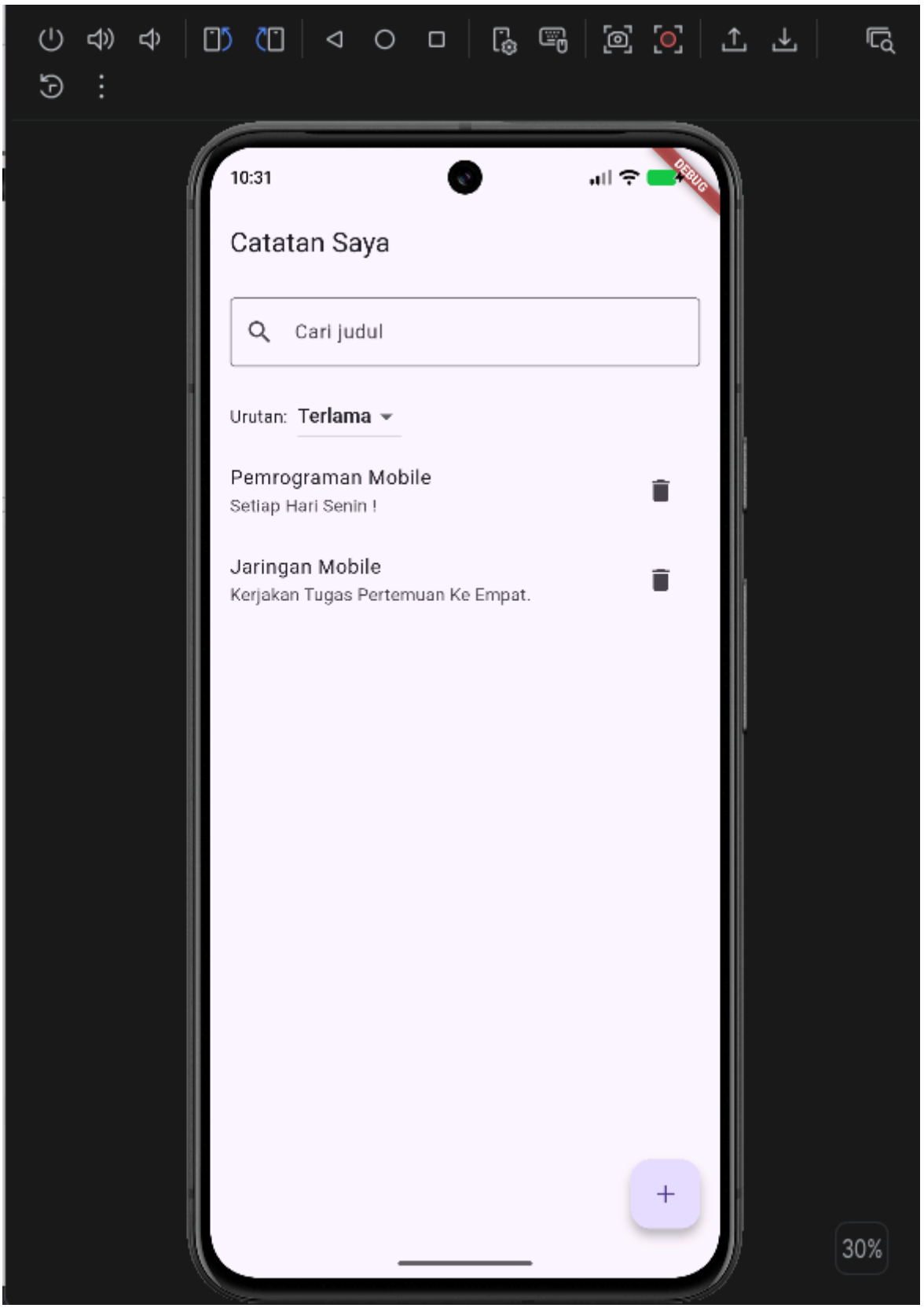
  @override
  Widget build(BuildContext context) {
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
```

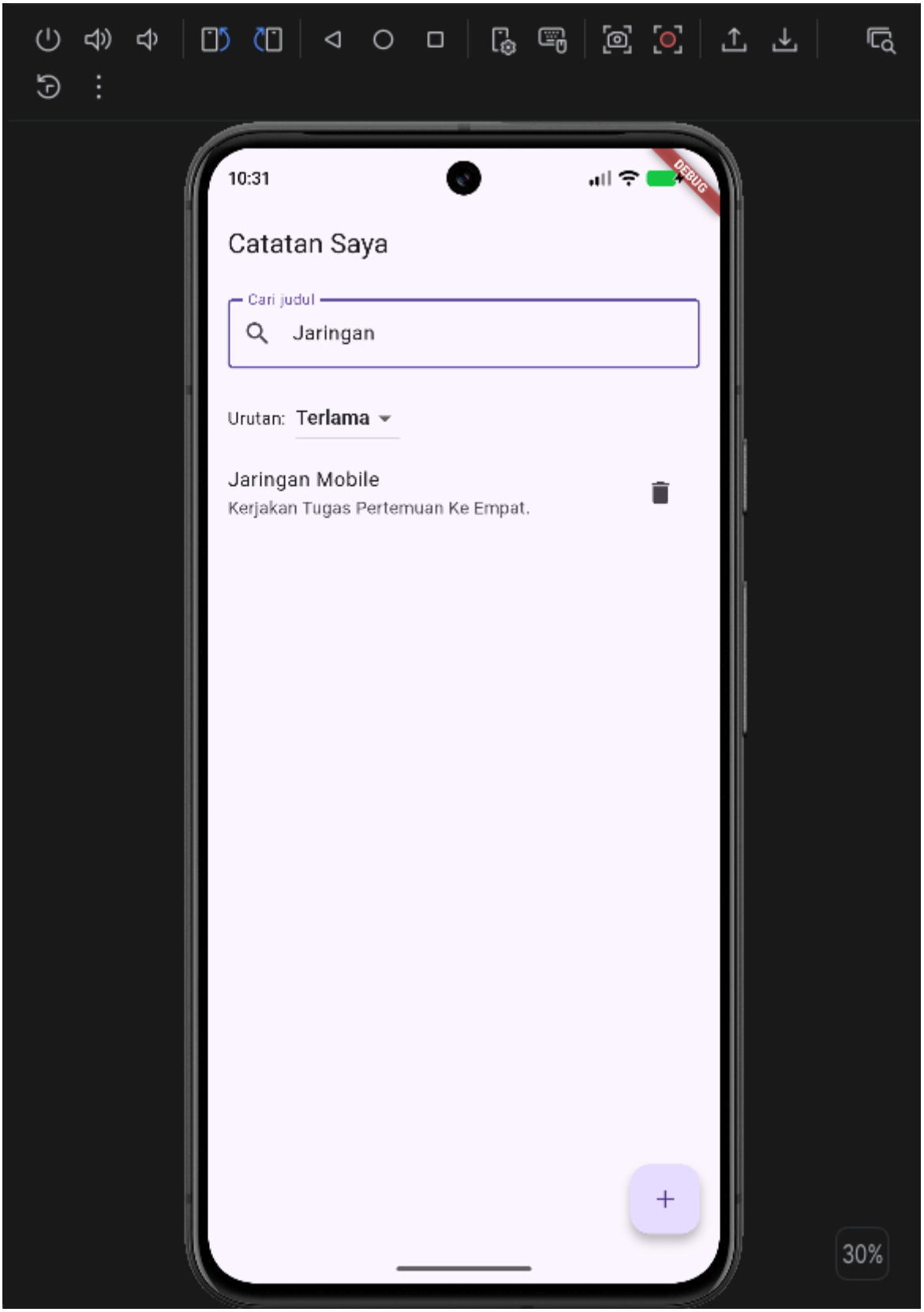
```dart
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ), InputDecoration
              validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ), TextFormField
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ), InputDecoration
              validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ), TextFormField
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: _simpan,
              child: const Text('Simpan'),
            ), ElevatedButton
          ],
        ), ListView
      ), Form
    ); Scaffold
  }
}
```

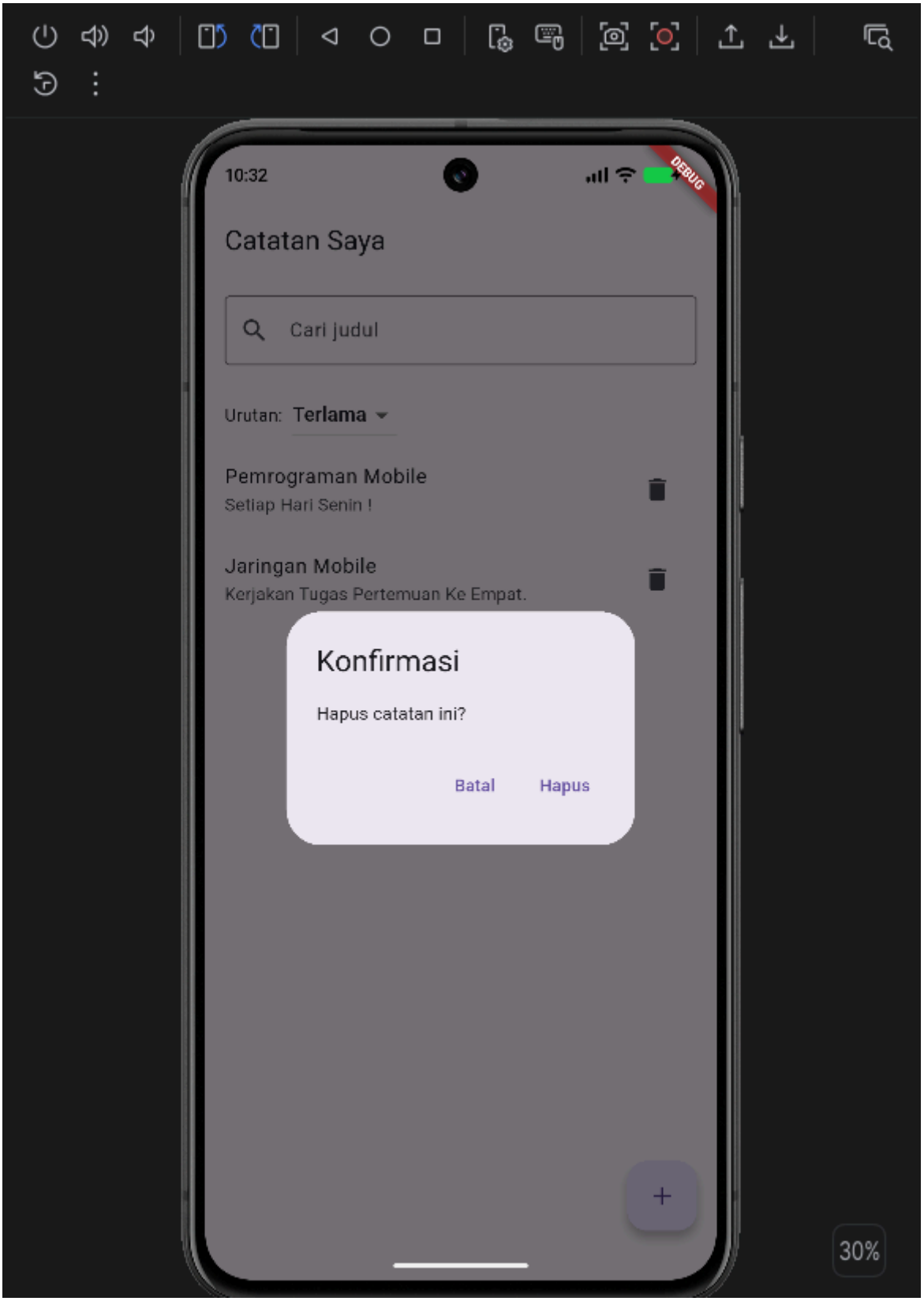

10:27
DEBUG
Catatan Saya
Cari judul
Urutan: Terbaru
Pemrograman Mobile
Setiap Hari Senin !
30%


Pixel 8 API 37.2
10:27
DEBUG
Catatan Baru
Judul
Judul wajib diisi
Isi catatan
Isi wajib diisi
Simpan
30%


10:31
DEBUG
Catatan Saya
Cari judul
Urutan: Terbaru
Jaringan Mobile
Kerjakan Tugas Pertemuan Ke Empat.
Pemrograman Mobile
Setiap Hari Senin !
30%


10:31
DEBUG
Catatan Saya
Cari judul
Urutan: Terlama
Pemrograman Mobile
Setiap Hari Senin !
Jaringan Mobile
Kerjakan Tugas Pertemuan Ke Empat.
30%


10:31
DEBUG
Catatan Saya
Cari judul
Jaringan
Urutan: Terlama
Jaringan Mobile
Kerjakan Tugas Pertemuan Ke Empat.
30%


10:32
DEBUG
Catatan Saya
Cari judul
Urutan: Terlama
Pemrograman Mobile
Setiap Hari Senin !
Jaringan Mobile
Kerjakan Tugas Pertemuan Ke Empat.
Konfirmasi
Hapus catatan ini?
Batal
Hapus
30%

5. Tugas

Buat aplikasi Pencatat Pengeluaran dengan ketentuan:

- Tabel pengeluaran: id, nama, jumlah (integer), kategori, dan tanggal.
- CRUD lengkap: daftar, tambah, ubah, hapus, dengan form berisi validasi (nama wajib, jumlah angka lebih dari 0, kategori dari dropdown).
- Menampilkan total pengeluaran di halaman utama (dihitung di Dart atau memakai SELECT SUM(jumlah)).
- Menyimpan satu pengaturan (misalnya mode gelap atau kategori filter terakhir) dengan shared_preferences.
- Data harus tetap ada setelah aplikasi ditutup dan dibuka kembali.
- Kumpulkan: tangkapan layar (daftar, form, dan bukti data bertahan setelah restart) serta berkas kode (atau tautan repositori).

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Pengeluaran {
  final int? id;
  final String nama;
  final int jumlah;
  final String kategori;
  final String tanggal;

  const Pengeluaran({
    this.id,
    required this.nama,
    required this.jumlah,
    required this.kategori,
    required this.tanggal,
  });

  Map<String, Object?> toMap() => {
    'id': id,
    'nama': nama,
    'jumlah': jumlah,
    'kategori': kategori,
    'tanggal': tanggal,
  };

  factory Pengeluaran.fromMap(Map<String, Object?> m) => Pengeluaran(
    id: m['id'] as int,
```

```
      nama: m['nama'] as String,                                    105 ^
      jumlah: m['jumlah'] as int,
      kategori: m['kategori'] as String,
      tanggal: m['tanggal'] as String,
    );
  }

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'pengeluaran.db');

    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE pengeluaran('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'nama TEXT NOT NULL, '
              'jumlah INTEGER NOT NULL, '
              'kategori TEXT NOT NULL, '
              'tanggal TEXT NOT NULL)',
        );
      },
    );

    return _db!;
```

```
  }                                                             105 ^

  static Future<int> tambah(Pengeluaran p) async {
    final db = await database;
    return db.insert('pengeluaran', p.toMap());
  }

  static Future<List<Pengeluaran>> semua() async {
    final db = await database;
    final rows = await db.query(
      'pengeluaran',
      orderBy: 'id DESC',
    );
    return rows.map(Pengeluaran.fromMap).toList();
  }

  static Future<int> ubah(Pengeluaran p) async {
    final db = await database;
    return db.update(
      'pengeluaran',
```

```
      p.toMap(),
      where: 'id = ?',
      whereArgs: [p.id],
    );
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete(
      'pengeluaran',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pencatat Pengeluaran',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ), ThemeData
      home: const PengeluaranPage(),
    ); MaterialApp
  }
}

class PengeluaranPage extends StatefulWidget {
  const PengeluaranPage({super.key});

  @override
  State<PengeluaranPage> createState() => _PengeluaranPageState();
}

class _PengeluaranPageState extends State<PengeluaranPage> {
  late Future<List<Pengeluaran>> _future;
  String _kategoriFilter = 'Semua';

  @override
  void initState() {
    super.initState();
    _muat();
```

105

```
    _muatPengaturan();
  }

  Future<void> _muatPengaturan() async {
    final prefs = await SharedPreferences.getInstance();
    final kategori = prefs.getString('kategoriFilter') ?? 'Semua';

    if (!mounted) return;

    setState(() {
      _kategoriFilter = kategori;
    });
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  Future<void> _simpanFilter(String kategori) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('kategoriFilter', kategori);

    if (!mounted) return;

    setState(() {
      _kategoriFilter = kategori;
    });
  }
```

```
  Future<void> _buka([Pengeluaran? pengeluaran]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(
        builder: (_) => FormPengeluaranPage(
          pengeluaran: pengeluaran,
        ),
      ),
    );

    if (!mounted) return;
    _muat();
  }

  Future<void> _hapus(Pengeluaran pengeluaran) async {
    final konfirmasi = await showDialog<bool>(
      context: context,
```

```
178          builder: (context) => AlertDialog(                    ✓105 ^
179            title: const Text('Konfirmasi'),
180            content: const Text('Hapus pengeluaran ini?'),
181            actions: [
182              TextButton(
183                onPressed: () => Navigator.pop(context, false),
184                child: const Text('Batal'),
185              ), TextButton
186              TextButton(
187                onPressed: () => Navigator.pop(context, true),
188                child: const Text('Hapus'),
189              ), TextButton
190            ],
191          ), AlertDialog
192        );
193
194        if (konfirmasi == true) {
195          await DbHelper.hapus(pengeluaran.id!);
196          _muat();
197        }
198      }
199
200      @override
201      Widget build(BuildContext context) {
202        return Scaffold(
203          appBar: AppBar(
204            title: const Text('Pengeluaran Saya'),
205          ), AppBar
206          body: FutureBuilder<List<Pengeluaran>>(
207            future: _future,
208            builder: (context, snapshot) {
209              if (snapshot.connectionState ==
210                  ConnectionState.waiting) {
211                return const Center(
212                  child: CircularProgressIndicator(),
213                ); Center
214              }
215
216              if (snapshot.hasError) {
217                return Center(
218                  child: Text('Galat: ${snapshot.error}'),
219                ); Center
220              }
221
222              final data = snapshot.data!;
223
224              final dataFilter = _kategoriFilter == 'Semua'
225                  ? data
226                  : data
227                  .where(
228                    (p) => p.kategori == _kategoriFilter,
```

```
229            )                                                    105
230                .toList();
231
232            final total = dataFilter.fold<int>(
233              0,
234                  (total, p) => total + p.jumlah,
235            );
236
237            return Column(
238              children: [
239                Padding(
240                  padding: const EdgeInsets.all(16),
241                  child: Card(
242                    child: Padding(
243                      padding: const EdgeInsets.all(16),
244                      child: Column(
245                        children: [
246                          const Text(
247                            'Total Pengeluaran',
248                          ), Text
249                          const SizedBox(height: 8),
250                          Text(
251                            'Rp $total',
252                            style: Theme.of(context)
253                                .textTheme
254                                .headlineSmall,
255                          ), Text
256                        ],
257                      ), Column
258                    ), Padding
```

```
260                ), Padding
261                Padding(
262                  padding: const EdgeInsets.symmetric(
263                    horizontal: 16,
264                  ), EdgeInsets.symmetric
265                  child: Row(
266                    children: [
267                      const Text('Kategori:'),
268                      const SizedBox(width: 12),
269                      DropdownButton<String>(
270                        value: _kategoriFilter,
271                        items: const [
272                          DropdownMenuItem(
273                            value: 'Semua',
274                            child: Text('Semua'),
275                          ), DropdownMenuItem
276                          DropdownMenuItem(
277                            value: 'Makanan',
278                            child: Text('Makanan'),
```

```
                    ), DropdownMenuItem
                    DropdownMenuItem(
                      value: 'Transportasi',
                      child: Text('Transportasi'),
                    ), DropdownMenuItem
                    DropdownMenuItem(
                      value: 'Belanja',
                      child: Text('Belanja'),
                    ), DropdownMenuItem
                    DropdownMenuItem(
                      value: 'Lainnya',
                      child: Text('Lainnya'),
                    ), DropdownMenuItem
                  ],
                  onChanged: (value) {
                    if (value != null) {
                      _simpanFilter(value);
                    }
                  },
                ), DropdownButton
              ],
            ), Row
          ), Padding
          Expanded(
            child: dataFilter.isEmpty
                ? const Center(
              child: Text(
```

```
                'Belum ada pengeluaran',
              ), Text
            ) Center
                : ListView.builder(
              itemCount: dataFilter.length,
              itemBuilder: (context, i) {
                final p = dataFilter[i];

                return ListTile(
                  title: Text(p.nama),
                  subtitle: Text(
                    '${p.kategori} • ${p.tanggal}',
                  ), Text
                  trailing: Row(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      Text('Rp ${p.jumlah}'),
                      IconButton(
                        icon: const Icon(
                          Icons.delete,
                        ), Icon
                        onPressed: () => _hapus(p),
```

```
                                ), IconButton
                              ],
                            ), Row
                            onTap: () => _buka(p),
                          ); ListTile
                        },
                      ), ListView.builder
                    ), Expanded
                  ],
                ); Column
              },
            ), FutureBuilder
            floatingActionButton: FloatingActionButton(
              onPressed: () => _buka(),
              child: const Icon(Icons.add),
            ), FloatingActionButton
          ); Scaffold
        }
      }

      class FormPengeluaranPage extends StatefulWidget {
        final Pengeluaran? pengeluaran;

        const FormPengeluaranPage({
          super.key,
          this.pengeluaran,
        });

        @override
        State<FormPengeluaranPage> createState() =>
```

105

```
            _FormPengeluaranPageState();
      }

      class _FormPengeluaranPageState
          extends State<FormPengeluaranPage> {
        final _formKey = GlobalKey<FormState>();

        late final TextEditingController _nama;
        late final TextEditingController _jumlah;
        late final TextEditingController _tanggal;

        String? _kategori;

        @override
        void initState() {
          super.initState();

          _nama = TextEditingController(
```

105

```
376         text: widget.pengeluaran?.nama ?? '',                    105
377       );
378
379       _jumlah = TextEditingController(
380         text: widget.pengeluaran?.jumlah.toString() ?? '',
381       );
382
383       _tanggal = TextEditingController(
384         text: widget.pengeluaran?.tanggal ??
385             DateTime.now().toString().substring(0, 10),
386       );  TextEditingController
387
388       _kategori = widget.pengeluaran?.kategori;
389     }
390
391     @override
392     void dispose() {
393       _nama.dispose();
394       _jumlah.dispose();
395       _tanggal.dispose();
396       super.dispose();
397     }
398
399     Future<void> _simpan() async {
400       if (!_formKey.currentState!.validate()) return;
401
402       final p = Pengeluaran(
403         id: widget.pengeluaran?.id,
404         nama: _nama.text.trim(),
405         jumlah: int.parse( jumlah.text.trim()),
```

```
406         kategori: _kategori!,                                   105
407         tanggal: _tanggal.text.trim(),
408       );
409
410       if (widget.pengeluaran == null) {
411         await DbHelper.tambah(p);
412       } else {
413         await DbHelper.ubah(p);
414       }
415
416       if (!mounted) return;
417
418       Navigator.pop(context);
419     }
420
421     @override
422     Widget build(BuildContext context) {
423       final baru = widget.pengeluaran == null;
424
```

```dart
    return Scaffold(
      appBar: AppBar(
        title: Text(
          baru
              ? 'Pengeluaran Baru'
              : 'Ubah Pengeluaran',
        ), Text
      ), AppBar
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ), InputDecoration
              validator: (v) =>
              (v == null || v.trim().isEmpty)
                  ? 'Nama wajib diisi'
                  : null,
            ), TextFormField
            const SizedBox(height: 12),
            TextFormField(
              controller: _jumlah,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
                labelText: 'Jumlah',
                border: OutlineInputBorder(),
              ), InputDecoration
              validator: (v) {
                if (v == null || v.trim().isEmpty) {
                  return 'Jumlah wajib diisi';
                }

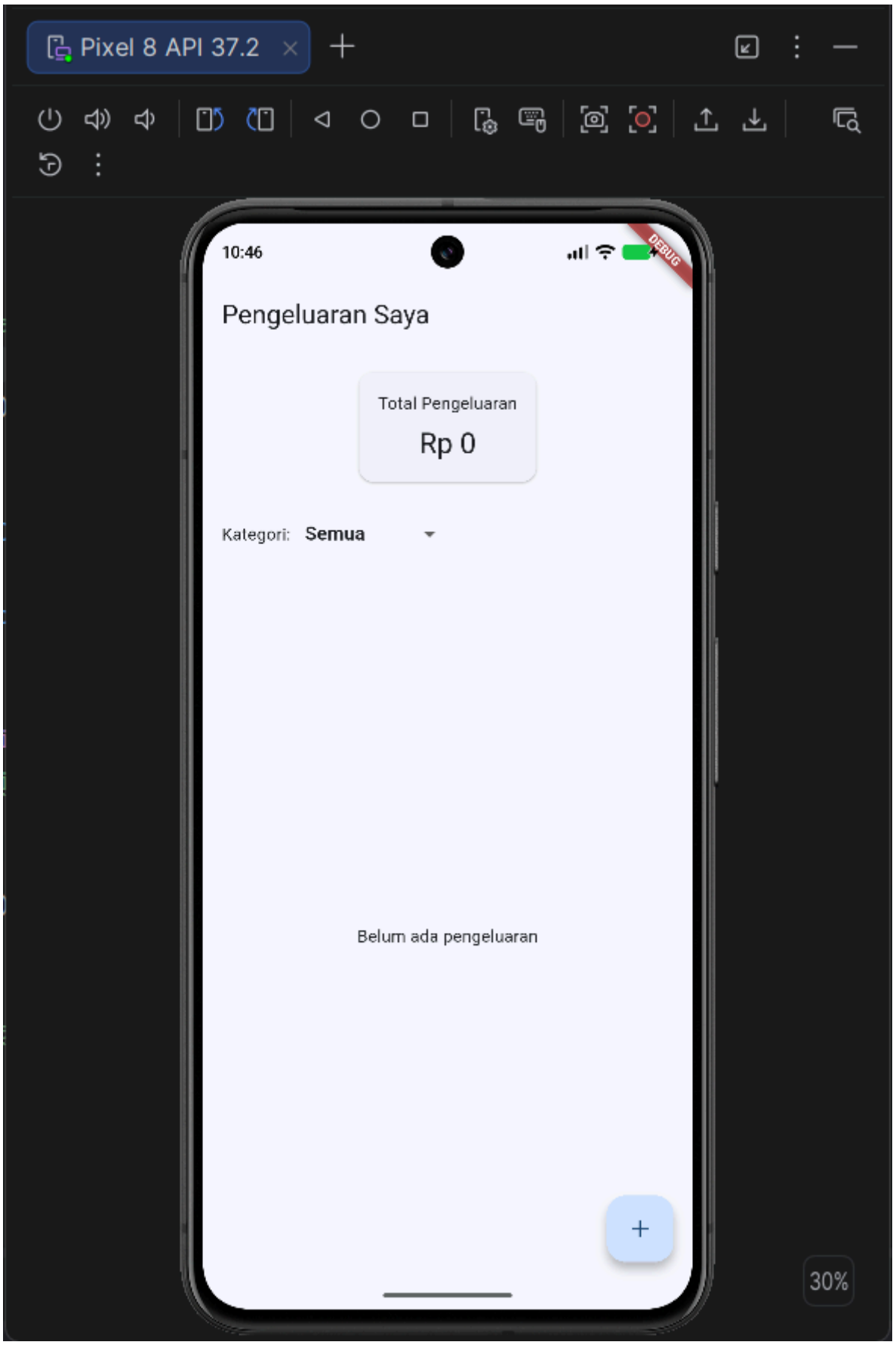
                final jumlah = int.tryParse(
                  v.trim(),
                );

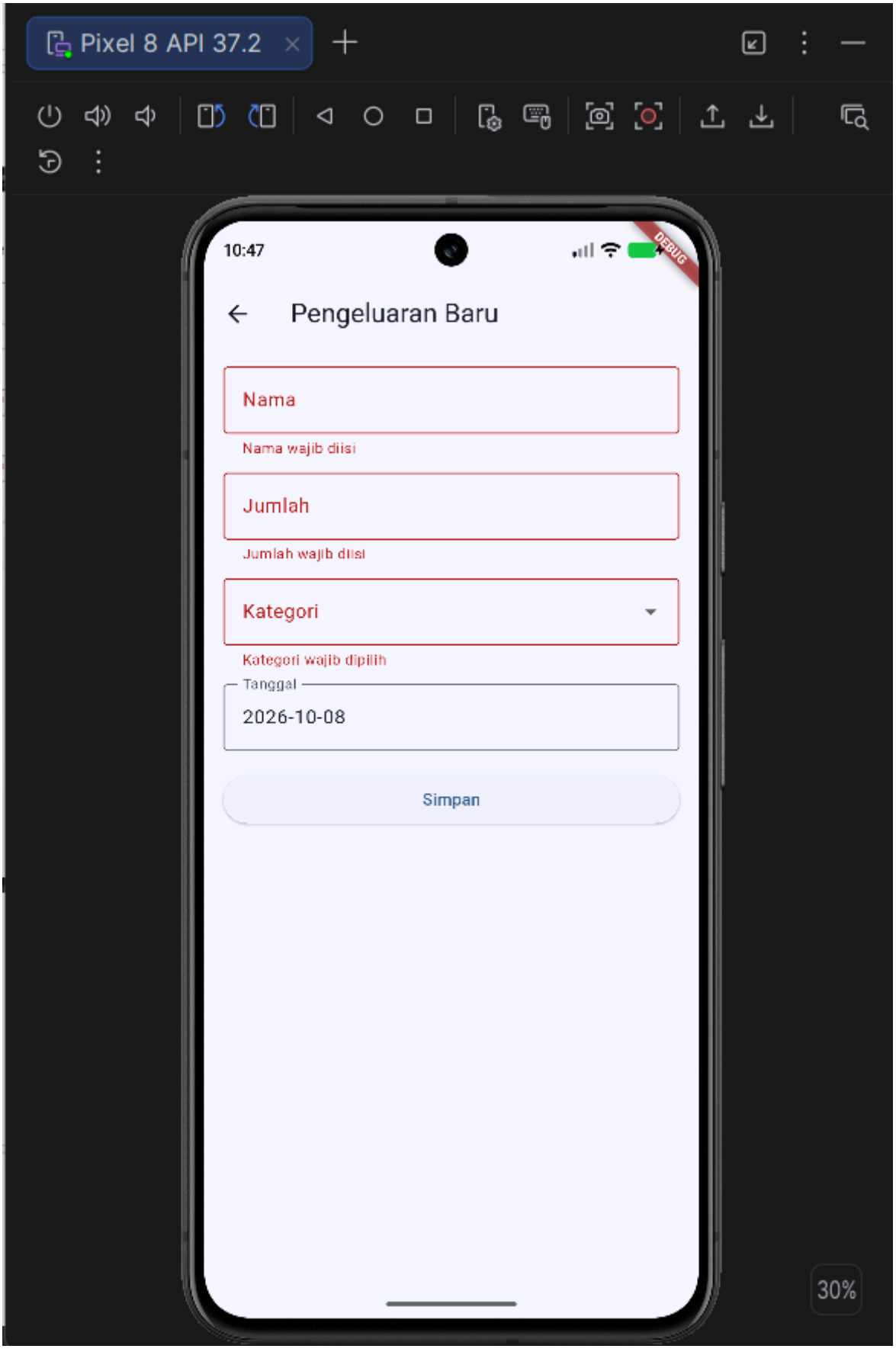
                if (jumlah == null) {
                  return 'Jumlah harus berupa angka';
                }

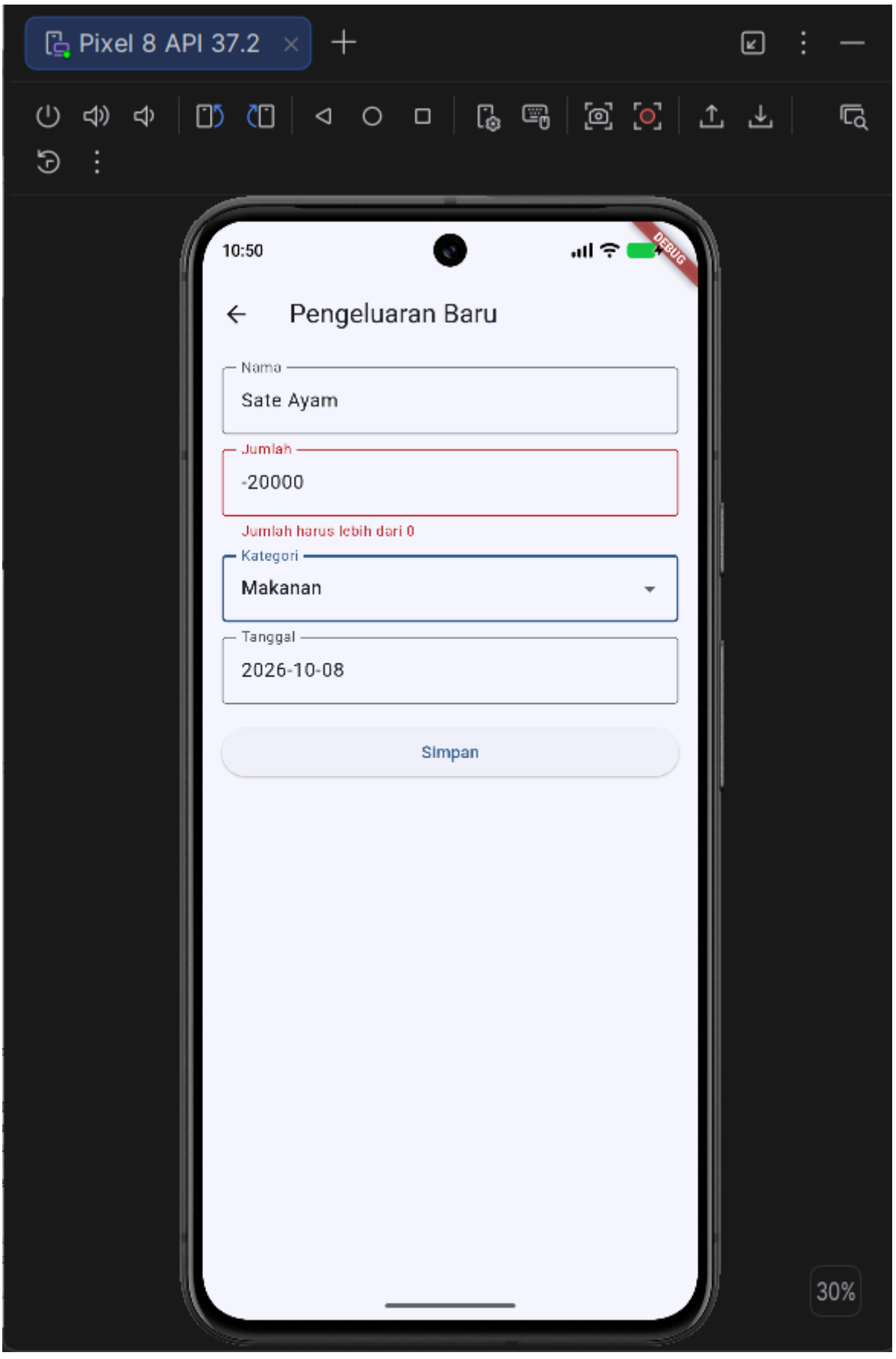
                if (jumlah <= 0) {
```

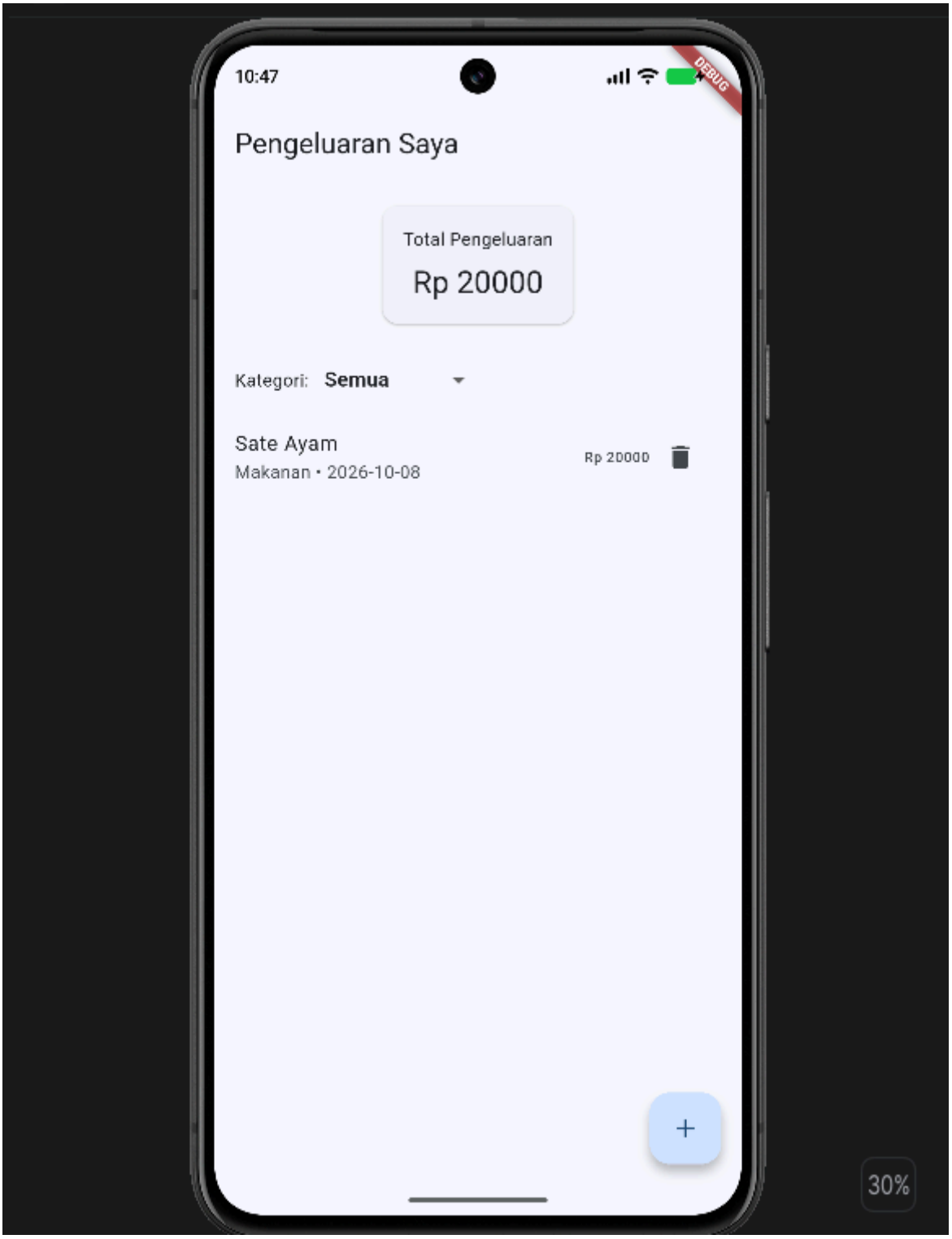
```
471                     return 'Jumlah harus lebih dari 0';
472                   }
473
474                   return null;
475                 },
476               ), TextFormField
477               const SizedBox(height: 12),
478               DropdownButtonFormField<String>(
479                 initialValue : _kategori,
480                 decoration: const InputDecoration(
481                   labelText: 'Kategori',
482                   border: OutlineInputBorder(),
483                 ), InputDecoration
484                 items: const [
485                   DropdownMenuItem(
486                     value: 'Makanan',
487                     child: Text('Makanan'),
488                   ), DropdownMenuItem
489                   DropdownMenuItem(
490                     value: 'Transportasi',
491                     child: Text('Transportasi'),
492                   ), DropdownMenuItem
493                   DropdownMenuItem(
494                     value: 'Belanja',
495                     child: Text('Belanja'),
496                   ), DropdownMenuItem
497                   DropdownMenuItem(
498                     value: 'Lainnya',
499                     child: Text('Lainnya'),
500                   ), DropdownMenuItem
```

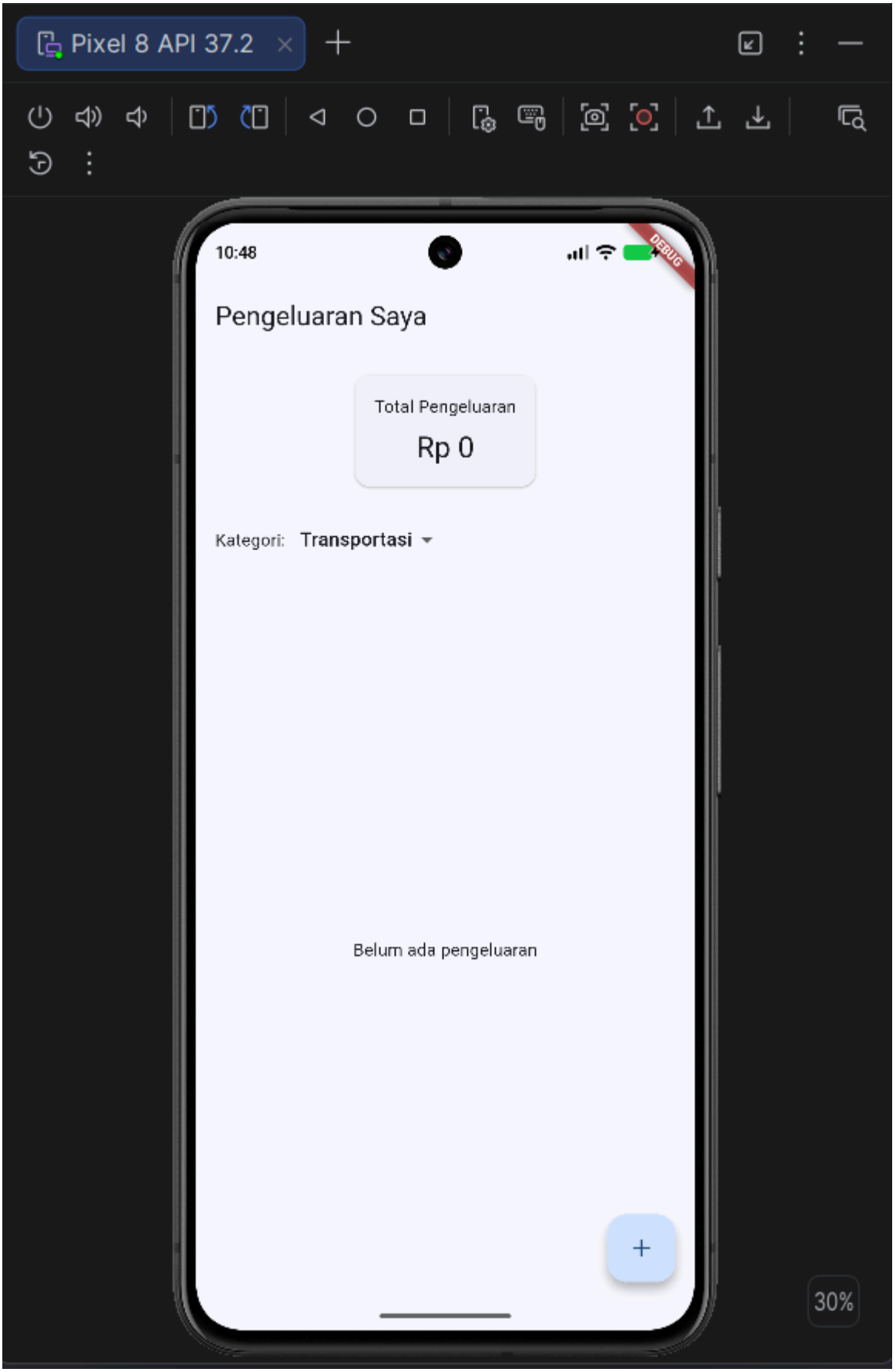
```
501                 ],
502                 onChanged: (value) {
503                   setState(() {
504                     _kategori = value;
505                   });
506                 },
507                 validator: (v) =>
508                 v == null ? 'Kategori wajib dipilih' : null,
509               ), DropdownButtonFormField
510               const SizedBox(height: 12),
511               TextFormField(
512                 controller: _tanggal,
513                 decoration: const InputDecoration(
514                   labelText: 'Tanggal',
515                   border: OutlineInputBorder(),
516                 ), InputDecoration
517                 validator: (v) =>
518                 (v == null || v.trim().isEmpty)
519                     ? 'Tanggal wajib diisi'
520                     : null,
521               ), TextFormField
522               const SizedBox(height: 16),
523               ElevatedButton(
524                 onPressed: _simpan,
525                 child: const Text('Simpan'),
526               ), ElevatedButton
527             ],
528           ), ListView
529         ), Form
530       ); Scaffold
```

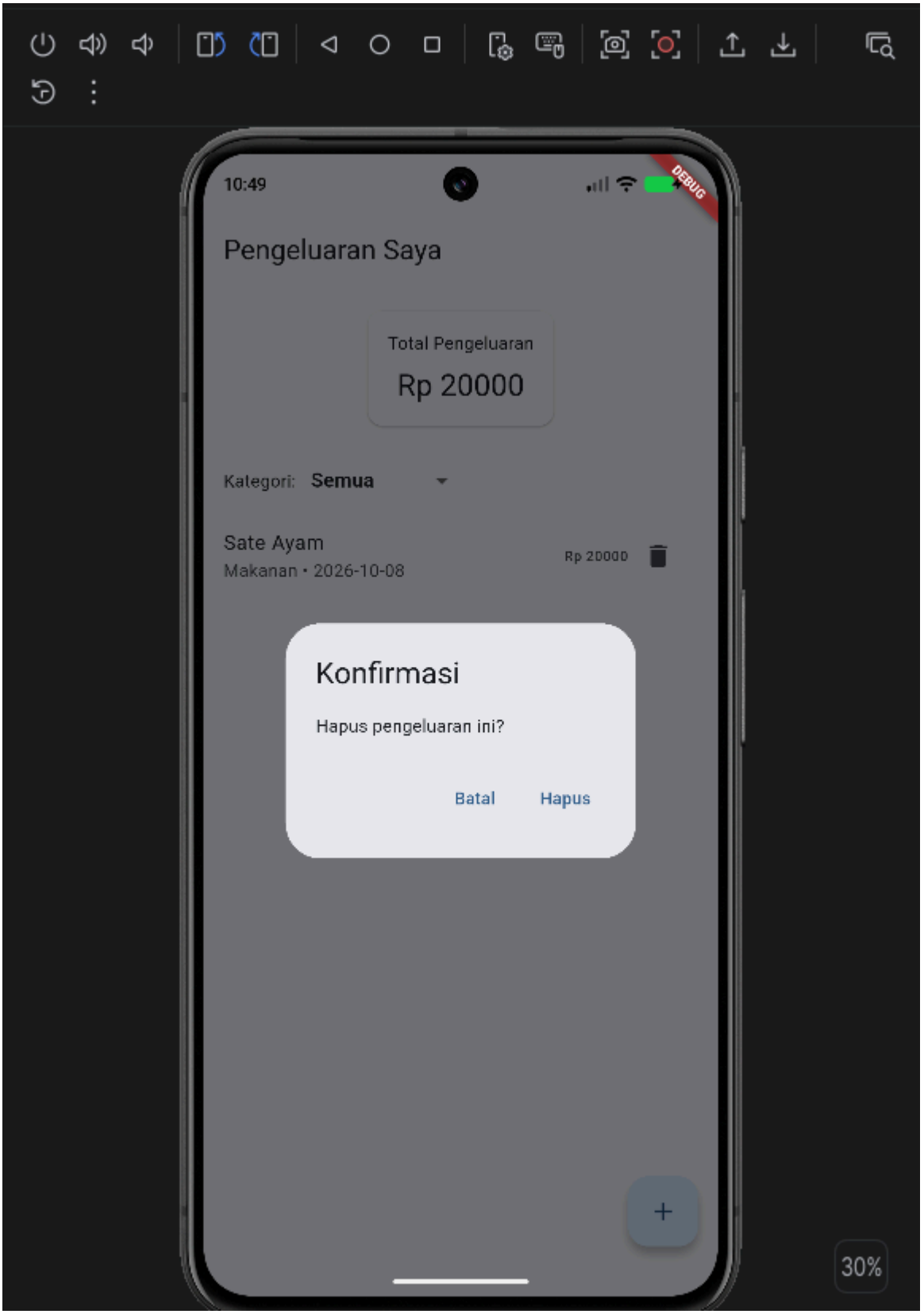

Pixel 8 API 37.2
10:46
DEBUG
Pengeluaran Saya
Total Pengeluaran
Rp 0
Kategori: Semua
Belum ada pengeluaran
30%


Pixel 8 API 37.2
10:47
DEBUG
Pengeluaran Baru
Nama
Nama wajib diisi
Jumlah
Jumlah wajib diisi
Kategori
Kategori wajib dipilih
Tanggal
2026-10-08
Simpan
30%


Pixel 8 API 37.2
10:50
DEBUG
Pengeluaran Baru
Nama
Sate Ayam
Jumlah
-20000
Jumlah harus lebih dari 0
Kategori
Makanan
Tanggal
2026-10-08
Simpan
30%


10:47
DEBUG
Pengeluaran Saya
Total Pengeluaran
Rp 20000
Kategori: Semua
Sate Ayam
Makanan • 2026-10-08
Rp 20000
30%


Pixel 8 API 37.2
10:48
DEBUG
Pengeluaran Saya
Total Pengeluaran
Rp 0
Kategori:
Transportasi
Belum ada pengeluaran
30%


10:49
DEBUG
Pengeluaran Saya
Total Pengeluaran
Rp 20000
Kategori: Semua
Sate Ayam
Makanan • 2026-10-08
Rp 20000
Konfirmasi
Hapus pengeluaran ini?
Batal
Hapus
30%

6. Pertanyaan Refleksi

1. Mengapa shared_preferences kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?

shared_preferences lebih cocok untuk menyimpan data kecil seperti pengaturan atau status sederhana. Untuk menyimpan ratusan catatan yang memiliki struktur data dan membutuhkan operasi tambah, baca, ubah, serta hapus, SQLite dengan sqflite lebih sesuai.

2. Mengapa query memakai placeholder ? dan whereArgs, bukan menyambung teks langsung?

Placeholder ? dan whereArgs digunakan agar nilai tidak langsung digabungkan ke query SQL. Cara ini lebih aman dan membantu mencegah SQL injection.

3. Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?

Daftar perlu dimuat ulang karena data mungkin sudah berubah setelah catatan ditambah atau diubah.

4. Apa fungsi onCreate dan onUpgrade, dan kapan masing-masing dijalankan?

onCreate digunakan ketika database pertama kali dibuat, misalnya untuk menjalankan CREATE TABLE. onUpgrade digunakan ketika versi database dinaikkan sehingga struktur database perlu diperbarui.

7. Catatan Pribadi

Persistence : Penyimpanan data agar tetap ada setelah aplikasi ditutup.

SharedPreferences : Penyimpanan sederhana berbentuk key-value.

setString() : Menyimpan data bertipe String.

getString() : Membaca data String.

setBool() : Menyimpan data Boolean.

getBool() : Membaca data Boolean.

setInt() : Menyimpan data Integer.

getInt() : Membaca data Integer.

SQLite : Database lokal untuk menyimpan data terstruktur.

db.insert() : Menambahkan data ke database.

db.query() : Mengambil data dari database.

db.update() : Mengubah data dalam database.

db.delete() : Menghapus data dari database.

toMap() : Mengubah object menjadi Map untuk disimpan ke database.

fromMap() : Mengubah data database menjadi object Dart.

onCreate : Dipanggil ketika database pertama kali dibuat.

onUpgrade : Digunakan ketika versi database dinaikkan untuk mengubah struktur database.

whereArgs : Mengirim nilai ke placeholder query SQL dengan lebih aman.